Grundlagen SSH

Remote verwaltung ohne aufwändiges GUI (IaaS kosten einsparung)

Telnet sendet im Klartext
↳ telnet host 80

RDP/AnyDesk benötigen mehr Recources.

Kryptografie:
Asymetrisch → Handshake & Authentifizierung Public Private keys rein
symetrisch → Nach erfolgreichem Handshake per Sitzungsschlüssel.

Keys:
ssh-keygen -t ed25519 -C "user@slaz.ch"
ssh-copy-id user@ipadrr → Public key copy
ssh -i ~/.ssh/custom_key user@ipadrr
Befehls ausführungen
ssh user@server 'sudo apt update && sudo apt get
scp ~/script.py user@server:/bin/ Lokal → Remote
scp user@server:/backup/dblogs.sql ~/backup.sql
Local Forwarding (-L) Weiterleitung von Lokalem port auf Zielport über ssh zz (Zugriff auf geschützte DB)
ssh -L localhost:1234:localhost:3456 user@ipadr
ssh -L 8800:10.0.0.5:80 user@server
Remote Forwarding (-R)
Sicherheitsrisiko: Server port lokal weiterleiten
Man in the Midlle Attachen.

→ An server Umleitung → Client
ssh -R server:8080:localhost:1234 user@ser

Valkey key value Store

Architektur: Valkey (Fork von Redis / Remote Dictionary Service) Verwaltet abstrakte Datentypen im In-Memory-Speicher unter Eindeutigen keys

Performance Arrays vs. Verkettete Listen
- Zugriff per Index: Array $O(1)$ vs Liste $O(n/2)$
- Einfügen am anfang: Array $O(n)$ (verschieben aller Elemente)
- Liste $O(1)$ (Pointer anpassung)
- Anfügen Am Ende: Beide $O(1)$ wenn Kapazität/tail-Pointer vorhanden ist.

Persistenzmodi im Vergleich:
- RDB (Snapshot): Periodische Zustandssicherung
- AOF (Append-only-File) Protokolliert jede schreib Transaktion
- RDB + AOF Höchste Sicherheit
- Cache Modus: Keine Persistenz, Nur In-Memory nutzung Hohe Performance, Nach Reboot weg

Valkey CLI & Auth

Ping → Pong
valkey-cli Startet shell
AUTH topsecret → Interaktive Passwort Authentifizierung
KEYS * → Alle keys | KEYS employee:*:Name
EXISTS key → bool | TYPE key → string, hash, list, set
FLUSHALL → Löscht alles | SAVE → RDB zwingen

Strings & Zahlen

SET user:100:name "John" | GET user:100:name
MSET k1 v1 k2 v2 | INCR user:100:age (Erhöhen um 1)
EXPIRE user:100:name 60 (TTL=60s) | TTL key

Listen: Linked List für Queues & Stacks

LPUSH todo "Aufgabe 1" "Aufgabe 2" | RPUSH todo "Task 3"
LRANGE todo 0 -1 (komplette Liste anzeigen)
LREM todo 1 "Aufgabe 1" (Entfernt 1 vorkommen)
RPOPLPUSH todo_pending todo_done (Atomares versch.)

Mengen (sets) ungeordnete eindeutige Elemente

SADD mammals Mensch Pferd Kuh Gorilla Fledermaus
SADD fourlegged Pferd Kuh Krokodil Eidechse

SINTER mammals fourlegged → Schnittmenge [Kuh, Pferd]
SDIFF mammals fourlegged → Keine Vierbeiner [Mensch, ...]
SUNION mammals fourlegged → vereinigung ohne Duplikate

Hashes (Maps) key value paare Pro Objekt
HSET endpoint:m346 url "https://cornmob.com" status 200 reqs 5
HGET endpoint:m346 url | HGETALL endpoint:m346
HKEYS endpoint:m346 | HDEL endpoint:m346 reqs

Export (CSV/JSON) & Passwort schutz

valkey-cli --csv HGETALL endpoint:m346
valkey-cli --json HGETALL endpoint:m346 > leten.json
config /etc/valkey/valkey.conf: requirepass topsecret
↳ Antruf valkey-cli -a topsecret PING

Frage 1: SSH Hybrid-Kryptografie
F: Wie nutzt SSH asymmetrische und symmetrische Kryptografie?
L: Asymmetrisch für Handshake & Client/Server-Auth; Symmetrisch für die Verschlüsselung der laufenden Sitzungsdaten.

Frage 2: SSH Remote Forwarding Risiko
F: Warum gilt ssh -R als Sicherheitsrisiko?
L: Anfragen an den Server werden auf den lokalen Entwickler-PC umgeleitet. Ermöglicht unbemerktes Ausführen von bösartigem Code.

Frage 3: SSH Key Transfer
F: Welcher Befehl kopiert den Public Key auf den Server?
L: ssh-copy-id user@server

Frage 4: Valkey Array vs. Liste Performance
F: Warum ist das Einfügen am Anfang einer Liste schneller als im Array?
L: Liste benötigt nur $O(1)$ Schritte (Pointer-Anpassung). Array benötigt $O(n)$ Schritte, da alle Elemente verschoben werden müssen.

Frage 5: Valkey Sets & Mengenoperationen
F: mammals=[Mensch, Pferd, Kuh], fourlegged=[Pferd, Kuh, Krokodil]. Was liefern SINTER, SDIFF und SUNION?
L: SINTER=[Pferd, Kuh] (Schnittmenge); SDIFF mammals fourlegged=[Mensch] SUNION=[Mensch, Pferd, Kuh, Krokodil].

S3 Object Storage

Speicherhierarchie:
- CPU-Register (<1ns, kB) → L1/L2 Cache (5ns, MB) → RAM (15ns, GB) → SSD (50µs, TB) → HDD (5ms, TB) → Tape (10–100s, TB)
- Faustregel schneller Speicher ist klein & teuer

S3 Konzepte:

- Flache Speicherstruktur
  - ↳ Buckets
  - ↳ Objects (Blob bis 5TB mit URL & Metadata)
- kein vendor Lock in S3 Standart.

S3 Speicherklassen:

Standart: Häufige zugriffe (SSD/HDD)
Standart-IA: seltene zugriffe Infrequent Access
Glacier: Langzeit Archivierung

RustFS S3 Server Setup:

nano ~/.profile → exaort RUSTFS_ACCESS_KEY=user
export RUSTFS_SECRET_KEY=Topsecret
export RUSTFS_REGION=bbzw && source ~/.profile
nohup rustfs server ~/s3-data & (web-GUI: 9001 API:9000)

Rust FS Client (rc) Commands

rc alias set local http://localhost:9000 user topsecret
rc ping local | rc admin info server local
rc bucket create local/Bachup
rc object list local/pictures
rc object copy
rc tag set local/pictures purpose=upload module=346
rc tag list --json local/pictures

S3cmd CLI Tool setup (~/.s3cfg)

config: host_base=localhost:9000, host_bucket=localhost:9000, use_https=false, access_key=user, secret_key=topsecret, region=BBZW
s3cmd mb s3://archive | s3cmd ls s3://pictures
s3cmd cp s3://bachup/file.png s3://archive/

S3fs FUSE Mounting

echo "user:topsecret" > ~/.passwd-s3fs
chmod 600 ~/.passwd-s3fs → Zwingend !
mkdir -p ~/s3-mount/pictures
s3fs pictures ~/s3-mount/pictures -o use_path_request_style, url=http://localhost:9000
mount | grep s3 | unmount ~/s3-mount/pictures